JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class MessageFormat

java.lang.Object
    java.text.Format
        java.text.MessageFormat

All Implemented Interfaces:

Serializable, Cloneable

---

public class **MessageFormat**
extends Format

MessageFormat provides a means to produce concatenated messages in a language-neutral way. Use this class to construct messages displayed for end users.

MessageFormat takes a set of objects, formats them, then inserts the formatted strings into the pattern at the appropriate places.

Note: MessageFormat differs from the other Format classes in that you create a MessageFormat object with one of its constructors (not with a getInstance style factory method). The factory methods aren't necessary because MessageFormat itself doesn't implement locale specific behavior. Any locale specific behavior is defined by the pattern that you provide as well as the subformats used for inserted arguments.

Patterns and Their Interpretation

MessageFormat uses patterns of the following form:

```
MessageFormatPattern:
        String
        MessageFormatPattern FormatElement String

FormatElement:
        { ArgumentIndex }
        { ArgumentIndex , FormatType }
        { ArgumentIndex , FormatType , FormatStyle }

FormatType:
        number
        dtf_date
        dtf_time
        dtf_datetime
        pre-defined DateTimeFormatter(s)
        date
```

```
        time
        choice
        list

FormatStyle:
        short
        medium
        long
        full
        integer
        currency
        percent
        compact_short
        compact_long
        or
        unit
        SubformatPattern
```

The `ArgumentIndex` value is a non-negative integer written using the digits `'0'` through `'9'`, and represents an index into the `arguments` array passed to the `format` methods or the result array returned by the `parse` methods.

Any constructor or method that takes a String pattern parameter will throw an `IllegalArgumentException` if the pattern contains an `ArgumentIndex` value that is equal to or exceeds an implementation limit.

The `FormatType` and `FormatStyle` values are used to create a `Format` instance for the format element. The following table shows how the values map to `Format` instances. These values are case-insensitive when passed to `applyPattern(String)`. Combinations not shown in the table are illegal. A *SubformatPattern* must be a valid pattern string for the `Format` subclass used.

Implementation Note:

In the reference implementation, the limit of `ArgumentIndex` is 10,000.

FormatType	FormatStyle	Subformat Created
(none)	*(none)*	`null`
`number`	*(none)*	`NumberFormat.getInstance(getLocale())`
	`integer`	`NumberFormat.getIntegerInstance(getLocale`
	`currency`	`NumberFormat.getCurrencyInstance(getLocal`
	`percent`	`NumberFormat.getPercentInstance(getLocale`
	`compact_short`	`NumberFormat.getCompactNumberInstance(get`
	`compact_long`	`NumberFormat.getCompactNumberInstance(get`

FormatType	FormatStyle	Subformat Created
	SubformatPattern	new DecimalFormat(subformatPattern, Decima
dtf_date	*(none)*	DateTimeFormatter.ofLocalizedDate(FormatS
	short	DateTimeFormatter.ofLocalizedDate(FormatS
	medium	DateTimeFormatter.ofLocalizedDate(FormatS
	long	DateTimeFormatter.ofLocalizedDate(FormatS
	full	DateTimeFormatter.ofLocalizedDate(FormatS
	SubformatPattern	DateTimeFormatter.ofPattern(subformatPatt
dtf_time	*(none)*	DateTimeFormatter.ofLocalizedTime(FormatS
	short	DateTimeFormatter.ofLocalizedTime(FormatS
	medium	DateTimeFormatter.ofLocalizedTime(FormatS
	long	DateTimeFormatter.ofLocalizedTime(FormatS
	full	DateTimeFormatter.ofLocalizedTime(FormatS
	SubformatPattern	DateTimeFormatter.ofPattern(subformatPatt
dtf_datetime	*(none)*	DateTimeFormatter.ofLocalizedDateTime(For
	short	DateTimeFormatter.ofLocalizedDateTime(For
	medium	DateTimeFormatter.ofLocalizedDateTime(For
	long	DateTimeFormatter.ofLocalizedDateTime(For
	full	DateTimeFormatter.ofLocalizedDateTime(For
	SubformatPattern	DateTimeFormatter.ofPattern(subformatPatt
pre-defined DateTimeFormatter(s)	*(none)*	The pre-defined DateTimeFormatter(s) are u ISO_LOCAL_DATE, ISO_OFFSET_DATE, ISO_DATE, I ISO_LOCAL_DATE_TIME, ISO_OFFSET_DATE_TIME, ISO_ORDINAL_DATE, ISO_WEEK_DATE, ISO_INSTAN
date	*(none)*	DateFormat.getDateInstance(DateFormat.DEF
	short	DateFormat.getDateInstance(DateFormat.SHO
	medium	DateFormat.getDateInstance(DateFormat.MED
	long	DateFormat.getDateInstance(DateFormat.LON
	full	DateFormat.getDateInstance(DateFormat.FUL
	SubformatPattern	new SimpleDateFormat(subformatPattern, ge
time	*(none)*	DateFormat.getTimeInstance(DateFormat.DEF
	short	DateFormat.getTimeInstance(DateFormat.SHO
	medium	DateFormat.getTimeInstance(DateFormat.MED
	long	DateFormat.getTimeInstance(DateFormat.LON
	full	DateFormat.getTimeInstance(DateFormat.FUL
	SubformatPattern	new SimpleDateFormat(subformatPattern, ge
choice	*SubformatPattern*	new ChoiceFormat(subformatPattern)

FormatType	FormatStyle	Subformat Created
list	*(none)*	ListFormat.getInstance(getLocale(), ListFo ListFormat.Style.FULL)
	or	ListFormat.getInstance(getLocale(), ListFo
	unit	ListFormat.getInstance(getLocale(), ListFo

Quoting Rules in Patterns

Within a *String*, a pair of single quotes can be used to quote any arbitrary characters except single quotes. For example, pattern string `"'{0}'"` represents string `"{0}"`, not a *FormatElement*. A single quote itself must be represented by doubled single quotes `''` throughout a *String*. For example, pattern string `"'{''}'"` is interpreted as a sequence of `'{` (start of quoting and a left curly brace), `''` (a single quote), and `}'` (a right curly brace and end of quoting), *not* `'{'` and `'}'` (quoted left and right curly braces): representing string `"{'}"`, *not* `"{}"`.

A *SubformatPattern* is interpreted by its corresponding subformat, and subformat-dependent pattern rules apply. For example, pattern string `"{1,number,$'#',##}"` (*SubformatPattern* with underline) will produce a number format with the pound-sign quoted, with a result such as: `"$#31,45"`. Refer to each `Format` subclass documentation for details.

Any unmatched quote is treated as closed at the end of the given pattern. For example, pattern string `"'{0}"` is treated as pattern `"'{0}'"`.

Any
curly braces within an unquoted pattern must be balanced. For example, `"ab {0} de"` and `"ab '}' de"` are valid patterns, but `"ab {0'}' de"`, `"ab } de"` and `"''{''"` are not.

Warning:

> The rules for using quotes within message format patterns unfortunately have shown to be somewhat confusing. In particular, it isn't always obvious to localizers whether single quotes need to be doubled or not. Make sure to inform localizers about the rules, and tell them (for example, by using comments in resource bundle source files) which strings will be processed by `MessageFormat`. Note that localizers may need to use single quotes in translated strings where the original version doesn't have them.

Usage Information

The following example demonstrates a general usage of `MessageFormat`. In internationalized programs, the message format pattern and other static strings will likely be obtained from resource bundles.

```
int planet = 7;
String event = "a disturbance in the Force";
String result = MessageFormat.format(
    "At {1,time} on {1,date}, there was {2} on planet {0,number,integer}.",
    planet, new GregorianCalendar(2053, Calendar.JULY, 3, 12, 30).getTime(), ev
```

`result` returns the following:

```
At 12:30:00 PM on Jul 3, 2053, there was a disturbance in the Force on plar
```

For more sophisticated patterns, `ChoiceFormat` can be used with `MessageFormat` to produce accurate forms for singular and plural:

```
MessageFormat msgFmt = new MessageFormat("The disk \"{0}\" contains {1,choice,0
Object[] args = {"MyDisk", fileCount};
String result = msgFmt.format(args);
```

`result` with different values for `fileCount`, returns the following:

```
The disk "MyDisk" contains no files.
The disk "MyDisk" contains one file.
The disk "MyDisk" contains 1,273 files.
```

Notes: As seen in the previous snippet, the string produced by a `ChoiceFormat` in `MessageFormat` is treated as special; occurrences of '{' are used to indicate subformats, and cause recursion. If a `FormatElement` is defined in the `ChoiceFormat` pattern, it will only be formatted according to the `FormatType` and `FormatStyle` pattern provided. The associated subformats of the top level `MessageFormat` will not be applied to the `FormatElement` defined in the `ChoiceFormat` pattern. If you create both a `MessageFormat` and `ChoiceFormat` programmatically (instead of using the string patterns), then be careful not to produce a format that recurses on itself, which will cause an infinite loop.

Formatting Date and Time

MessageFormat provides patterns that support the date/time formatters in the `java.time.format` and `java.text` packages. Consider the following three examples, with a date of 11/16/2023:

1) a *date* `FormatType` with a *full* `FormatStyle`,

```
Object[] arg = {new GregorianCalendar(2023, Calendar.NOVEMBER, 16).getTime()};
var fmt = new MessageFormat("The date was {0,date,full}");
fmt.format(arg); // returns "The date was Thursday, November 16, 2023"
```

2) a *dtf_date* `FormatType` with a *full* `FormatStyle`,

```
Object[] arg = {LocalDate.of(2023, 11, 16)};
var fmt = new MessageFormat("The date was {0,dtf_date,full}");
fmt.format(arg); // returns "The date was Thursday, November 16, 2023"
```

3) an *ISO_LOCAL_DATE* `FormatType`,

```
Object[] arg = {LocalDate.of(2023, 11, 16)};
var fmt = new MessageFormat("The date was {0,ISO_LOCAL_DATE}");
fmt.format(arg); // returns "The date was 2023-11-16"
```

Parsing

When a single argument is parsed more than once in the string, the last match will be the final result of the parsing. For example,

```
MessageFormat mf = new MessageFormat("{0,number,#.##}, {0,number,#.#}");
Object[] objs = {Double.valueOf(3.1415)};
String result = mf.format( objs );
// result now equals "3.14, 3.1"
objs = mf.parse(result, new ParsePosition(0));
// objs now equals {Double.valueOf(3.1)}
```

Likewise, parsing with a `MessageFormat` object using patterns containing multiple occurrences of the same argument would return the last match. For example,

```
MessageFormat mf = new MessageFormat("{0}, {0}, {0}");
String forParsing = "x, y, z";
Object[] objs = mf.parse(forParsing, new ParsePosition(0));
// objs now equals {new String("z")}
```

Synchronization

Message formats are not synchronized. It is recommended to create separate format instances for each thread. If multiple threads access a format concurrently, it must be synchronized externally.

Since:

1.1

See Also:

Locale, Format, NumberFormat, DecimalFormat, DecimalFormatSymbols, ChoiceFormat, DateFormat, SimpleDateFormat, DateTimeFormatter, Serialized Form

Nested Class Summary

Nested Classes

Modifier and Type	Class	Description
static class	MessageFormat.Field	Defines constants that are used as attribute keys in the AttributedCharacterIterator returned from MessageFormat.formatToCharacterIterator.

Constructor Summary

Constructors

Constructor	Description
MessageFormat(String pattern)	Constructs a MessageFormat for the default FORMAT locale and the specified pattern.
MessageFormat(String pattern, Locale locale)	Constructs a MessageFormat for the specified locale and pattern.

Method Summary

All Methods | **Static Methods**

Modifier and Type	Method	Description
void		

	applyPattern(String pattern)	Sets the pattern used by this message format.
Object	**clone**()	Creates and returns a copy of this object.
boolean	**equals**(Object obj)	Compares the specified object with this MessageFormat for equality.
final StringBuffer	**format**(Object[] arguments, StringBuffer result, FieldPosition pos)	Formats an array of objects and appends the MessageFormat's pattern, with format elements replaced by the formatted objects, to the provided StringBuffer.
final StringBuffer	**format**(Object arguments, StringBuffer result, FieldPosition pos)	Formats an array of objects and appends the MessageFormat's pattern, with format elements replaced by the formatted objects, to the provided StringBuffer.
static String	**format**(String pattern, Object... arguments)	Creates a MessageFormat with the given pattern and uses it to format the given arguments.
AttributedCharacterI	**formatToCharacterIterato**(Object arguments)	Formats an array of objects and inserts them into the MessageFormat's pattern, producing an AttributedCharacterIterator.
Format[]	**getFormats**()	Gets the formats used for the format elements in the previously set pattern string.
Format[]	**getFormatsByArgumentInde**	Gets the formats used for the values passed

		into `format` methods or returned from `parse` methods.
`Locale`	**`getLocale`**`()`	Gets the locale that's used when creating or comparing subformats.
`int`	**`hashCode`**`()`	Returns the hash code value for this `MessageFormat`.
`Object[]`	**`parse`**`(String source)`	Parses text from the beginning of the given string to produce an object array.
`Object[]`	**`parse`**`(String source, ParsePosition pos)`	Parses the string.
`Object`	**`parseObject`**`(String source, ParsePosition pos)`	Parses text from a string to produce an object array.
`void`	**`setFormat`**`(int formatElementIndex, Format newFormat)`	Sets the format to use for the format element with the given format element index within the previously set pattern string.
`void`	**`setFormatByArgumentIndex`**`(int argumentIndex, Format newFormat)`	Sets the format to use for the format elements within the previously set pattern string that use the given argument index.
`void`	**`setFormats`**`(Format[] newFormats)`	Sets the formats to use for the format elements in the previously set pattern string.
`void`		Sets the formats to use for the values passed

	setFormatsByArgumentInde(Format[] newFormats)	into format methods or returned from parse methods.
void	**setLocale**(Locale locale)	Sets the locale to be used when creating or comparing subformats.
String	**toPattern**()	Returns a String pattern adhering to the patterns section that represents the current state of this MessageFormat.
String	**toString**()	Returns a string identifying this MessageFormat, for debugging.

Methods declared in class Format

format, parseObject

Methods declared in class Object

finalize, getClass, notify, notifyAll, wait, wait, wait

Constructor Details

MessageFormat

```
public MessageFormat(String pattern)
```

Constructs a MessageFormat for the default FORMAT locale and the specified pattern. The constructor first sets the locale, then parses the pattern and creates a list of subformats for the format elements contained in it. Patterns and their interpretation are specified in the class description.

Parameters:

pattern - the pattern for this message format

Throws:

IllegalArgumentException - if the pattern is invalid

NullPointerException - if pattern is null

MessageFormat

```
public MessageFormat(String pattern,
                     Locale locale)
```

Constructs a MessageFormat for the specified locale and pattern. The constructor first sets the locale, then parses the pattern and creates a list of subformats for the format elements contained in it. Patterns and their interpretation are specified in the class description.

Implementation Requirements:

The default implementation throws a NullPointerException if locale is null either during the creation of the MessageFormat object or later when format() is called by a MessageFormat instance with a null locale and the implementation utilizes a locale-dependent subformat.

Parameters:

pattern - the pattern for this message format

locale - the locale for this message format

Throws:

IllegalArgumentException - if the pattern is invalid

NullPointerException - if pattern is null or locale is null and the implementation uses a locale-dependent subformat.

Since:

1.4

Method Details

setLocale

```
public void setLocale(Locale locale)
```

Sets the locale to be used when creating or comparing subformats. This affects subsequent calls

- to the applyPattern and toPattern methods if format elements specify a format type and therefore have the subformats created in the applyPattern method, as well as
- to the format and formatToCharacterIterator methods if format elements do not specify a format type and therefore have the subformats created in the formatting methods.

Subformats that have already been created are not affected.

Parameters:

`locale` - the locale to be used when creating or comparing subformats

getLocale

```
public Locale getLocale()
```

Gets the locale that's used when creating or comparing subformats.

Returns:

the locale used when creating or comparing subformats

applyPattern

```
public void applyPattern(String pattern)
```

Sets the pattern used by this message format. The method parses the pattern and creates a list of subformats for the format elements contained in it. Patterns and their interpretation are specified in the class description.

Parameters:

`pattern` - the pattern for this message format

Throws:

`IllegalArgumentException` - if the pattern is invalid

`NullPointerException` - if `pattern` is `null`

toPattern

```
public String toPattern()
```

Returns a String pattern adhering to the `patterns section` that represents the current state of this `MessageFormat`. The string is constructed from internal information and therefore does not necessarily equal the previously applied pattern. The order of `FormatStyle` matching is not guaranteed. That is, a `FormatStyle` produced may not be equivalent to the corresponding style passed, in the instance that multiple styles are equivalent.

Implementation Requirements:

The implementation in `MessageFormat` returns a string that, when passed to a `MessageFormat()` constructor or `applyPattern()`, produces an instance that is semantically equivalent to this instance. If a subformat cannot be converted to a String pattern, the `FormatType` and `FormatStyle` will be omitted from the `FormatElement`.

Returns:

a String pattern adhering to the `patterns section` that represents the current state of this `MessageFormat`

setFormatsByArgumentIndex

```
public void setFormatsByArgumentIndex(Format[] newFormats)
```

Sets the formats to use for the values passed into `format` methods or returned from `parse` methods. The indices of elements in `newFormats` correspond to the argument indices used in the previously set pattern string. The order of formats in `newFormats` thus corresponds to the order of elements in the `arguments` array passed to the `format` methods or the result array returned by the `parse` methods.

If an argument index is used for more than one format element in the pattern string, then the corresponding new format is used for all such format elements. If an argument index is not used for any format element in the pattern string, then the corresponding new format is ignored. If fewer formats are provided than needed, then only the formats for argument indices less than `newFormats.length` are replaced.

Parameters:

`newFormats` - the new formats to use

Throws:

`NullPointerException` - if `newFormats` is null

Since:

1.4

setFormats

```
public void setFormats(Format[] newFormats)
```

Sets the formats to use for the format elements in the previously set pattern string. The order of formats in `newFormats` corresponds to the order of format elements in the pattern string.

If more formats are provided than needed by the pattern string, the remaining ones are ignored. If fewer formats are provided than needed, then only the first `newFormats.length` formats are replaced.

Since the order of format elements in a pattern string often changes during localization, it is generally better to use the `setFormatsByArgumentIndex` method, which assumes an order of formats corresponding to the order of elements in the `arguments` array passed to the `format` methods or the result array returned by the `parse` methods.

Parameters:

`newFormats` - the new formats to use

Throws:

NullPointerException - if newFormats is null

setFormatByArgumentIndex

```
public void setFormatByArgumentIndex(int argumentIndex,
                                     Format newFormat)
```

Sets the format to use for the format elements within the previously set pattern string that use the given argument index. The argument index is part of the format element definition and represents an index into the arguments array passed to the format methods or the result array returned by the parse methods.

If the argument index is used for more than one format element in the pattern string, then the new format is used for all such format elements. If the argument index is not used for any format element in the pattern string, then the new format is ignored.

Parameters:

argumentIndex - the argument index for which to use the new format

newFormat - the new format to use

Since:

1.4

setFormat

```
public void setFormat(int formatElementIndex,
                      Format newFormat)
```

Sets the format to use for the format element with the given format element index within the previously set pattern string. The format element index is the zero-based number of the format element counting from the start of the pattern string.

Since the order of format elements in a pattern string often changes during localization, it is generally better to use the setFormatByArgumentIndex method, which accesses format elements based on the argument index they specify.

Parameters:

formatElementIndex - the index of a format element within the pattern

newFormat - the format to use for the specified format element

Throws:

ArrayIndexOutOfBoundsException - if formatElementIndex is equal to or larger than the number of format elements in the pattern string

getFormatsByArgumentIndex

```
public Format[] getFormatsByArgumentIndex()
```

Gets the formats used for the values passed into `format` methods or returned from `parse` methods. The indices of elements in the returned array correspond to the argument indices used in the previously set pattern string. The order of formats in the returned array thus corresponds to the order of elements in the `arguments` array passed to the `format` methods or the result array returned by the `parse` methods.

If an argument index is used for more than one format element in the pattern string, then the format used for the last such format element is returned in the array. If an argument index is not used for any format element in the pattern string, then null is returned in the array.

Returns:

the formats used for the arguments within the pattern

Since:

1.4

getFormats

```
public Format[] getFormats()
```

Gets the formats used for the format elements in the previously set pattern string. The order of formats in the returned array corresponds to the order of format elements in the pattern string.

Since the order of format elements in a pattern string often changes during localization, it's generally better to use the `getFormatsByArgumentIndex` method, which assumes an order of formats corresponding to the order of elements in the `arguments` array passed to the `format` methods or the result array returned by the `parse` methods.

Returns:

the formats used for the format elements in the pattern

format

```
public final StringBuffer format(Object[] arguments,
                                 StringBuffer result,
                                 FieldPosition pos)
```

Formats an array of objects and appends the `MessageFormat`'s pattern, with format elements replaced by the formatted objects, to the provided `StringBuffer`.

The text substituted for the individual format elements is derived from the current subformat of the format element and the `arguments` element at the format element's argument index as indicated by the first matching line of the following table. An argument is *unavailable* if `arguments` is `null` or has fewer than argumentIndex+1 elements.

Subformat	Argument	Formatted Text
any	*unavailable*	`"{" + argumentIndex + "}"`
	`null`	`"null"`
`instanceof ChoiceFormat`	*any*	`subformat.format(argument).indexOf('{') >= 0 ? (new MessageFormat(subformat.format(argument), getLocale())).format(argument) : subformat.format(argument)`
`!= null`	*any*	`subformat.format(argument)`
`null`	`instanceof Number`	`NumberFormat.getInstance(getLocale()).format(argument)`
	`instanceof Date`	`DateFormat.getDateTimeInstance(DateFormat.SHORT, DateFormat.SHORT, getLocale()).format(argument)`
	`instanceof String`	`argument`
	any	`argument.toString()`

If `pos` is non-null, and refers to `Field.ARGUMENT`, the location of the first formatted string will be returned.

Parameters:

`arguments` - an array of objects to be formatted and substituted.

`result` - where text is appended.

`pos` - keeps track on the position of the first replaced argument in the output string.

Returns:

the string buffer passed in as `result`, with formatted text appended

Throws:

`IllegalArgumentException` - if an argument in the `arguments` array is not of the type expected by the format element(s) that use it.

`NullPointerException` - if `result` is `null` or if the `MessageFormat` instance that calls this method has locale set to null, and the implementation uses a locale-dependent subformat.

format

```
public static String format(String pattern,
                            Object... arguments)
```

Creates a MessageFormat with the given pattern and uses it to format the given arguments. This method returns a string that would be equal to the string returned by

```
(new MessageFormat(pattern)).format(arguments, new
StringBuffer(), null).toString()
```

Parameters:

`pattern` - the pattern string

`arguments` - object(s) to format

Returns:

the formatted string

Throws:

`IllegalArgumentException` - if the pattern is invalid, or if an argument in the `arguments` array is not of the type expected by the format element(s) that use it.

`NullPointerException` - if `pattern` is `null`

format

```
public final StringBuffer format(Object arguments,
                                 StringBuffer result,
                                 FieldPosition pos)
```

Formats an array of objects and appends the `MessageFormat`'s pattern, with format elements replaced by the formatted objects, to the provided `StringBuffer`. This is equivalent to

```
format((Object[]) arguments, result, pos)
```

Specified by:

`format` in class `Format`

Parameters:

`arguments` - an array of objects to be formatted and substituted.

`result` - where text is appended.

`pos` - keeps track on the position of the first replaced argument in the output string.

Returns:

the string buffer passed in as `toAppendTo`, with formatted text appended

Throws:

`IllegalArgumentException` - if an argument in the `arguments` array is not of the type expected by the format element(s) that use it.

`NullPointerException` - if `result` is `null` or if the `MessageFormat` instance that calls this method has locale set to null, and the implementation uses a locale-dependent subformat.

formatToCharacterIterator

```
public AttributedCharacterIterator formatToCharacterIterator
(Object arguments)
```

Formats an array of objects and inserts them into the `MessageFormat`'s pattern, producing an `AttributedCharacterIterator`. You can use the returned `AttributedCharacterIterator` to build the resulting String, as well as to determine information about the resulting String.

The text of the returned `AttributedCharacterIterator` is the same that would be returned by

```
format(arguments, new StringBuffer(), null).toString()
```

In addition, the `AttributedCharacterIterator` contains at least attributes indicating where text was generated from an argument in the `arguments` array. The keys of these attributes are of type `MessageFormat.Field`, their values are `Integer` objects indicating the index in the `arguments` array of the argument from which the text was generated.

The attributes/value from the underlying `Format` instances that `MessageFormat` uses will also be placed in the resulting `AttributedCharacterIterator`. This allows you to not only find where an argument is placed in the resulting String, but also which fields it contains in turn.

Overrides:

`formatToCharacterIterator` in class `Format`

Parameters:

`arguments` - an array of objects to be formatted and substituted.

Returns:

AttributedCharacterIterator describing the formatted value.

Throws:

`NullPointerException` - if `arguments` is null.

`IllegalArgumentException` - if an argument in the `arguments` array is not of the type expected by the format element(s) that use it.

Since:

1.4

parse

```
public Object[] parse(String source,
                      ParsePosition pos)
```

Parses the string.

Caveats: The parse may fail in a number of circumstances. For example:

- If one of the arguments does not occur in the pattern.
- If the format of an argument loses information, such as with a choice format where a large number formats to "many".
- Does not yet handle recursion (where the substituted strings contain {n} references.)
- Will not always find a match (or the correct match) if some part of the parse is ambiguous. For example, if the pattern "{1},{2}" is used with the string arguments {"a,b", "c"}, it will format as "a,b,c". When the result is parsed, it will return {"a", "b,c"}.
- If a single argument is parsed more than once in the string, then the later parse wins.

When the parse fails, use ParsePosition.getErrorIndex() to find out where in the string the parsing failed. The returned error index is the starting offset of the sub-patterns that the string is comparing with. For example, if the parsing string "AAA {0} BBB" is comparing against the pattern "AAD {0} BBB", the error index is 0. When an error occurs, the call to this method will return null. If the source is null, return an empty array.

Parameters:

`source` - the string to parse

`pos` - the parse position

Returns:

an array of parsed objects

Throws:

`NullPointerException` - if `pos` is `null` for a non-null `source` string.

parse

```
public Object[] parse(String source)
              throws ParseException
```

Parses text from the beginning of the given string to produce an object array. The method may not use the entire text of the given string.

See the parse(String, ParsePosition) method for more information on message parsing.

Parameters:

source - A String whose beginning should be parsed.

Returns:

An Object array parsed from the string.

Throws:

ParseException - if the beginning of the specified string cannot be parsed.

parseObject

```
public Object parseObject(String source,
                          ParsePosition pos)
```

Parses text from a string to produce an object array.

The method attempts to parse text starting at the index given by pos. If parsing succeeds, then the index of pos is updated to the index after the last character used (parsing does not necessarily use all characters up to the end of the string), and the parsed object array is returned. The updated pos can be used to indicate the starting point for the next call to this method. If an error occurs, then the index of pos is not changed, the error index of pos is set to the index of the character where the error occurred, and null is returned.

See the parse(String, ParsePosition) method for more information on message parsing.

Specified by:

parseObject in class Format

Parameters:

source - A String, part of which should be parsed.

pos - A ParsePosition object with index and error index information as described above.

Returns:

An Object array parsed from the string. In case of error, returns null.

Throws:

NullPointerException - if pos is null.

clone

```
public Object clone()
```

Creates and returns a copy of this object.

Overrides:

`clone` in class `Format`

Returns:

a clone of this instance.

See Also:

`Cloneable`

equals

```
public boolean equals(Object obj)
```

Compares the specified object with this `MessageFormat` for equality. Returns true if the object is also a `MessageFormat` and the two formats would format any value the same.

Overrides:

`equals` in class `Object`

Implementation Requirements:

This method performs an equality check with a notion of class identity based on `getClass()`, rather than `instanceof`. Therefore, in the equals methods in subclasses, no instance of this class should compare as equal to an instance of a subclass.

Parameters:

`obj` - object to be compared for equality

Returns:

`true` if the specified object is equal to this `MessageFormat`

See Also:

`Object.equals(Object)`

hashCode

```
public int hashCode()
```

Returns the hash code value for this `MessageFormat`.

Overrides:

`hashCode` in class `Object`

Implementation Requirements:

This method calculates the hash code value using the value returned by `toPattern()`.

Returns:

the hash code value for this `MessageFormat`

See Also:

`Object.hashCode()`

toString

```
public String toString()
```

Returns a string identifying this `MessageFormat`, for debugging.

Overrides:

`toString` in class `Object`

Returns:

a string identifying this `MessageFormat`, for debugging

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.